UNIVERSITY of BRADFORD
Faculty of Engineering & Informatics

COS6012-B: Concurrent and Distributed Systems

Synchronisation in Java

(Basics)

Dr Daniele Scrimieri

Outline

- Basic thread synchronization
- Synchronizing a method
- Using conditions in synchronized code
- Synchronizing a block of code with a lock

Concepts

- **Problem:** more than one execution thread shares a resource, e.g. read or write the same data structure or have access to the same file or database.
- **Race conditions** occur when different threads have access to the same shared resource at the same time. The final result depends on the order of the execution of threads, and most of the time, it is incorrect.
- **Interference**: destructive update, caused by the arbitrary interleaving of read and write actions.
- Interference bugs are extremely difficult to locate. The general solution is to give methods **mutually exclusive access to shared objects.**
- Mutual exclusion (or **mutex**) can be modelled as atomic actions.
- **Critical section:** a block of code that accesses a shared resource and can't be executed by more than one thread at the same time.

Mutual Exclusion in Java

- Concurrent activations of a method in Java can be made mutually exclusive by prefixing the method with the keyword **`synchronized`**, which uses a lock on the object.
- Basic synchronization mechanisms offered by the Java language:
  - The **`synchronized`** keyword
  - The **`Lock`** interface and its implementations:
    - ReentrantLock
    - ReentrantReadWriteLock.ReadLock
    - ReentrantReadWriteLock.WriteLock
- More advanced classes implementing monitors, semaphores, barriers, thread-pools, collections that are thread-safe
  - In next lectures: Semaphore, CyclicBarrier etc.

Java Concurrency Utilities Package

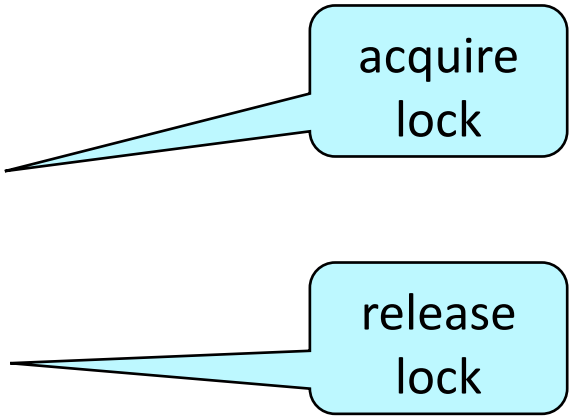
- Java 5 introduced a package of advanced concurrency utilities in **`java.util.concurrent`**, later extended in Java 8.
- This includes many additional, explicit mechanisms such as atomic variables, a task scheduling framework (for thread and thread pool instantiation and control), and synchronizers such as semaphores (next lecture), mutexes, barriers and explicit locks with timeout.

- **`synchronized:`** implicit lock associated with each object, block structured and recursive (reentrant)
- **`Lock`** interface: explicit lock objects, with methods *lock(), unlock(), tryLock()* with optional timeout.
- **`ReentrantLock:`** implements Lock (optionally *fair*), reentrant with methods *lock(), unlock(), tryLock()* with optional timeout, …

Java synchronized statement

- Concurrent activations of a method in Java can be made mutually exclusive by prefixing the method with the keyword **synchronized**, which uses a lock on the object.

```
... synchronized void myMethod() {
      count++;
}
```



- Access to an object may also be made mutually exclusive by using the synchronized statement:

```
synchronized (object) {
      // statements
}
```

Java synchronized statement

- Java associates a **lock** with every object.
- The Java compiler inserts code to acquire the lock before executing the body of the synchronized method and code to release the lock before the method returns.
- Concurrent threads are blocked until the lock is released.
- To ensure mutually exclusive access to an object, all object methods should be synchronized.

Example 1: RaceCondition

```
public class RaceCondition extends Thread {

    private static int total = 0;
    public static final int THREADS = 4;
    public static final int COUNT = 100;

    public void run() {
        for(int i = 0; i < COUNT; i++)
            total++;
    }
```

Example adapted from
https://start-concurrent.github.io/chunked/chap13.html#ch14-synchronization

Example 1: RaceCondition

```
public static void main(String[] args) {
    RaceCondition[] threads = new RaceCondition[THREADS];
    for(int i = 0; i < THREADS; i++) {
            threads[i] = new RaceCondition();
            threads[i].start();
    }
    try {
       for(int i = 0; i < THREADS; i++)
                threads[i].join();
        }
    catch(InterruptedException e) {
          e.printStackTrace();
    }
    System.out.println("Total :\t" + total);
  }
}
```

Final Results?

THREADS = 4;

- COUNT = 100
  - Total: 400 (okay)
- COUNT = 1000
  - Total : 4000 (okay)
  - Total : 4000 (okay)
  - …
- COUNT = 10000
  - Total : 20775 (not okay)
  - Total : 24008 (not okay)
  - Total : 33682 (not okay)

THREADS = 10;

- COUNT = 100
  - Total : 1000 (okay)
- COUNT = 1000
  - Total : 10000 (okay)
  - Total : 10000 (okay)
  - …
  - Total : 9804 (not okay)
- COUNT = 10000
  - Total : 23516 (not okay)
  - Total : 27533 (not okay)
  - Total : 29955 (not okay)

Adding Delays to Help Testing

```
public void run() {
    for(int i = 0; i < COUNT; i++){
        try {
            TimeUnit.MILLISECONDS.sleep(1);
            total++;
        } catch (InterruptedException ex) {
            ex.printStackTrace();
        }
    }
}
```

THREADS = 4;

COUNT = 100;

- SLEEP 1 ms
  - Total : 344
- SLEEP 5 ms
  - Total : 327

Bugs… Why, where ?

- Statement **total++** in the for loop inside the run() method.
  - **total++** is not a single (atomic) step.
- It is shorthand for **total = total + 1** which could be written as
  - `temp = total;`
  - `total = temp + 1;`
- One thread might get as far as storing **`total`** into a temporary location, but then it runs out of time, allowing the next thread in the schedule to run.
- When that's the case, this next thread may do a series of increments to **`total`** which are all overwritten when the first thread runs again.
- Because the first thread had an old value of counter stored in **temp**, adding 1 to **`temp`** has the effect of ignoring increments that happened in the interim.

Solution: synchronized Method

```
public void run() {
    for (int i = 0; i < COUNT; i++) {
            try {
                TimeUnit.MILLISECONDS.sleep(1);
            } catch (InterruptedException ex) {
                ex.printStackTrace();
            }
            increment();
        }
    }
public static synchronized void increment() {
    total++;
}
```

Using synchronized for More Critical Sections

```
private static Object lock1 = new Object();
private static Object lock2 = new Object();
. . .
synchronized(lock1) {
    // Do dangerous thing 1
}
// Do safe things
synchronized(lock2) {
    // Do dangerous thing 2, unrelated to
dangerous thing 1
}
```

Example2: Synchronized Buffer

```
public class Buffer {
   public final static int SIZE = 10;
   private Object[] objects = new Object[SIZE];
   private int count = 0;

   public synchronized void addItem(Object object)
      throws InterruptedException {
          while (count == SIZE)
              wait();
          objects[count] = object;
          count++;
          notifyAll();
      }
```

Example2: Synchronized Buffer (c'tnd)

```
public synchronized Object removeItem()
    throws InterruptedException {
        while(count == 0)
            wait();
        count--;
        Object object = objects[count];
        notifyAll();
        return object;
    }
}
```

Exercise: Write a Main class, with a main method that uses a Buffer. Create 2 threads, one that puts (random) objects in the buffer and one that consumes them.

Condition Synchronization in Java

- **`public final void notify()`**

Wakes up a single thread that is waiting on this object's wait set.

- **`public final void notifyAll()`**

Wakes up all threads that are waiting on this object's wait set.

- **`public final void wait()`**
  `throws InterruptedException`

Waits to be notified by another thread. The waiting thread releases the synchronization lock associated with the object (monitor). When notified, the thread must wait to reacquire the object (monitor) before resuming execution.

Condition Synchronization in Java

- Java relies heavily on a tool called **monitor** which hides some of the details of enforcing mutual exclusion from the user.
- So… what is a monitor? (will be detailed in lecture 3)
  - *thread-safe object*
  - *synchronization construct that allows threads to have both mutual exclusion and the ability to wait (block) for a certain condition*
- When writing our code we need to identify *threads* and *monitors*
  - **thread** - **active** entity which initiates actions
  - **monitor** - **passive** entity which responds to actions.
- Mutual exclusion has more approaches, other than monitors.

Condition Synchronization in Java

```
public synchronized void method() throws
InterruptedException  {
      while (!cond)
            wait();
      // modify monitor data
      notifyAll();
   }
```

- The while loop is necessary to retest the condition *cond* to ensure that *cond* is indeed satisfied when it re-enters the monitor.
- **`notifyAll()`** is necessary to awaken other thread(s) that may be waiting to enter the monitor now that the monitor data has been changed.

Synchronizing a Block of Code with a Lock

- More powerful and flexible mechanism than the synchronized keyword.
- It's based on the Lock interface and classes that implement it (e.g. ReentrantLock).
- It allows the structuring of synchronized blocks in a more flexible way: more complex structures to implement in the critical section.
- Additional functionalities over the synchronized keyword. One of the new functionalities is implemented by the tryLock() method. This method tries to get the control of the lock and if it can't, because it's used by other thread, it returns the lock.
- Better performance than the synchronized keyword.

Lock API

- **void lock()** – acquire the lock if it's available; if the lock isn't available a thread gets blocked until the lock is released
- **void lockInterruptibly()** – this is similar to the lock(), but it allows the blocked thread to be interrupted and resume the execution through a thrown java.lang.InterruptedException
- **boolean tryLock()** – this is a non-blocking version of lock() method; it attempts to acquire the lock immediately, return true if locking succeeds
- **boolean tryLock(long timeout, TimeUnit timeUnit)** – this is similar to tryLock(), except it waits up the given timeout before giving up trying to acquire the Lock
- **void unlock()** – unlocks the Lock instance

Using ReentrantLock

```
class MyClass {
   private final ReentrantLock lock =
      new ReentrantLock();
   // ...
   public void myMethod() {
     lock.lock();  // block until condition holds
     try {
       // ... method body -  Critical section!
     }
     finally {
       lock.unlock();
     }
   }
}
```

Working With Conditions

- **Condition:** provides the ability for a thread to wait for some condition to occur while executing the critical section.
- This can occur when a thread acquires the access to the critical section but doesn't have the necessary condition to perform its operation.
  - For example, a consumer thread can get access to the lock of a shared buffer, which still doesn't have any data to consume.
- Traditionally Java provides wait(), notify() and notifyAll() methods for thread intercommunication.
- Conditions have similar mechanisms, but in addition, we can specify multiple conditions.

BoundedBuffer (1/3)

```
class BoundedBuffer {
   final Lock lock = new ReentrantLock();
   final Condition notFull  = lock.newCondition();
   final Condition notEmpty = lock.newCondition();

   final Object[] items = new Object[100];
   int putptr, takeptr, count;
```

A Condition instance is intrinsically bound to a lock. To obtain a Condition instance for a particular Lock instance use its **newCondition()** method.

https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/Condition.html

BoundedBuffer (2/3)

```
public void put(Object x) throws
InterruptedException {
    lock.lock();
    try {
      while (count == items.length)
        notFull.await();
      items[putptr] = x;
      if (++putptr == items.length)
          putptr = 0;
      ++count;
      notEmpty.signal();
    } finally {
      lock.unlock();
    }
  }
```

BoundedBuffer (3/3)

```
public Object take() throws
InterruptedException {
    lock.lock();
    try {
      while (count == 0)
        notEmpty.await();
      Object x = items[takeptr];
      if (++takeptr == items.length)
          takeptr = 0;
      --count;
      notFull.signal();
      return x;
    } finally {
      lock.unlock();
    }
  }
}
```

Summary

- Tips for finding problems with race conditions, testing and debugging concurrent programs:
  - More operations
  - More threads
  - More delays
- Synchronized keyword
- Lock interface and its implementation
  - Lock, ReentrantLock, Condition
- Monitor: passive entity, synchronized methods
- Threads: active entities, using the monitor

Acknowledgments (and resources for further reading)

Books

- J. Fernandez Gonzalez (2012), Java 7 Concurrency Cookbook link, Packt Publishing
- B. Wittman, T. Korb, A. Mathur (2013), Start Concurrent: An Introduction to Problem Solving in Java with a Focus on Concurrency, Purdue University Press. Chapter 14: https://start-concurrent.github.io/chunked/chap13.html#ch14-synchronization
- Magee & Kramer (2006). Concurrency: State Models and Java Programs (2nd edition), John Wiley. Online slides https://www.doc.ic.ac.uk/~jnm/book/

Online tutorials

- https://docs.oracle.com/javase/tutorial/essential/concurrency/
- Jakob Jenkov. Java Concurrency Utilities http://tutorials.jenkov.com/java-concurrency/synchronized.html
- https://www.baeldung.com/java-concurrent-locks